

CS339: Abstractions and Paradigms for Programming

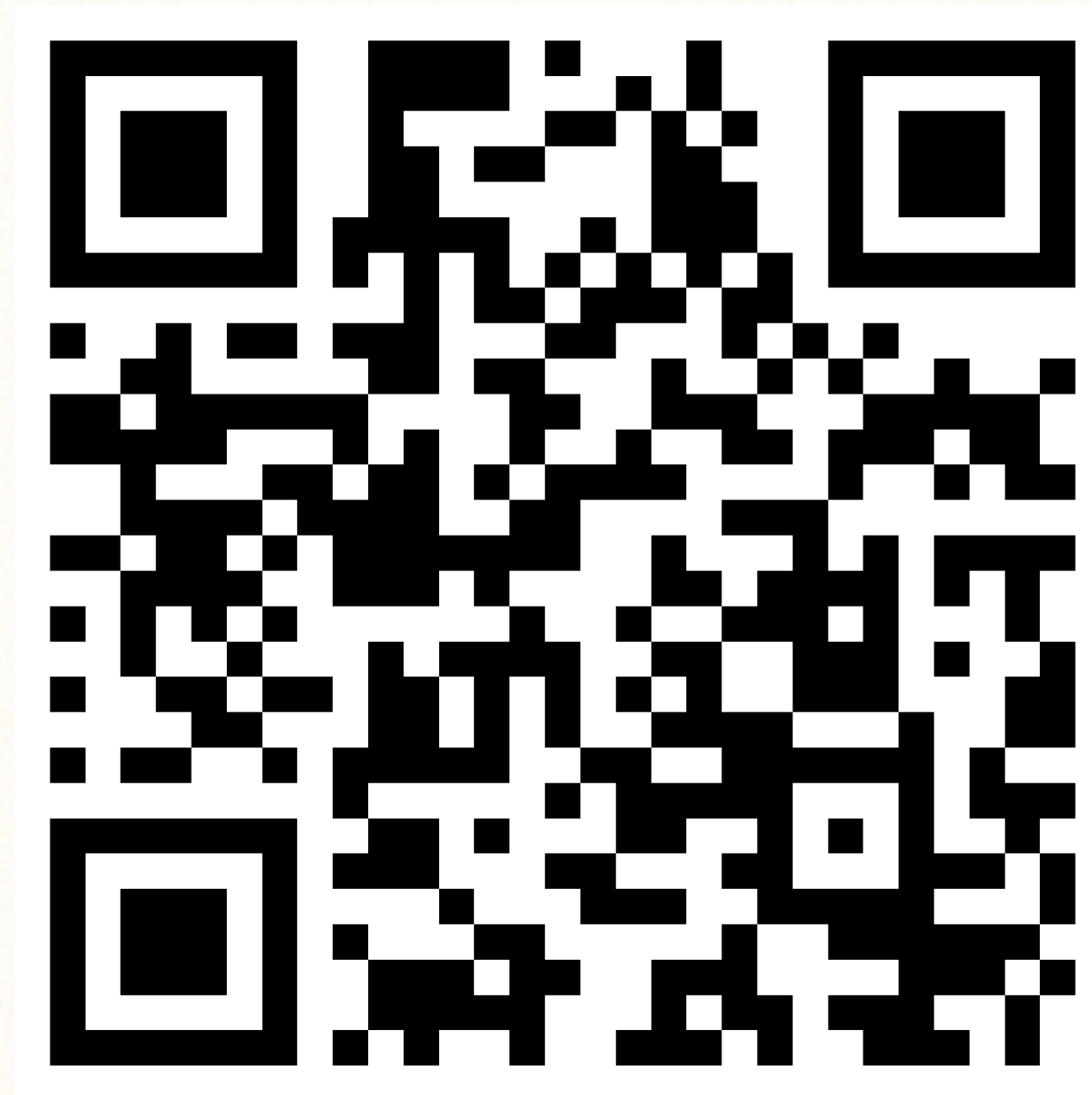
Visitors and Generics

Manas Thakur
CSE, IIT Bombay



Autumn 2026

Pop Quiz 5



Lab Midsem on Thursday

- **Most important instruction:** Follow the instructions written on the exam paper.
- Use the scripts provided to **validate** your answers before submission. Keep saving. Move to the next question if one takes too much time.
- **Syllabus:** Both Scheme and Java, covered until Thursday's lecture.
- We would share the seating arrangement (SL*, Basement) by Thursday morning.
- Arrive in the labs by 2:15 PM. Exam begins asap after that.
- Duration: 2 hours (with usual extra-time rules).
- Stay honest. Cell phones and smart devices have to be kept out. Bring ID cards.



Recap: Java Primer

- Variables and methods can be defined only inside classes
- Classes can be instantiated to create objects
- A class can extend another class and inherit its fields
- A class can extend another class and override its methods
- Parent class references can point to child class objects, but not vice versa
- A class can extend only one class but implement any number of interfaces
- All methods are virtual by default — resolved based on the “receiver” object
- Java class libraries provide useful data-structure implementations such as `ArrayList`, `LinkedList`, `Vector`, `HashSet`, `HashMap`, etc.



Let's look at an OO problem

- **Goal:** We want to sum the elements of an integer linked list:

```
interface List {}  
  
class Nil implements List {}  
  
class Cons implements List {  
    int head;  
    List tail;  
}
```



Approach 1: Instanceof checks and Typecasts

```
List l; ...
int sum = 0;
boolean proceed = true;
while (proceed) {
    if (l instanceof Nil) {
        proceed = false;
    }
    else if (l instanceof Cons) {
        sum = sum + ((Cons) l).head;
        l = ((Cons) l).tail;
    }
}
```

- **Bad:** Typecasts and instanceof checks are ugly (and expensive).
- **Good:** No need to touch Nil and Cons (abstraction principles respected).



Approach 2: Utilize OO style

```
interface List {  
    int sum();  
}  
class Nil implements List {  
    public int sum() { return 0; }  
}  
class Cons implements List {  
    int head;  
    List tail;  
    public int sum() {  
        return head + tail.sum();  
    }  
}
```

- **Good:** No typecasts and instanceof.
- **Bad:** Original classes to be recompiled for each new operation.



Approach 3: Visitor Pattern

- Divide code into an object structure and a **Visitor**.
- Insert an `accept` method in each structure class. Each `accept` method takes a **Visitor** as an argument.
- A **Visitor** contains a `visit` method for each structure class.

```
interface List {  
    void accept(Visitor v);  
}  
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```



Approach 3: Visitor Pattern

- The purpose of accept methods is to invoke the visit method in the Visitor that can handle the current object.

```
class Nil implements List {  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}  
  
class Cons implements List {  
    int head;  
    List tail;  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```



Approach 3: Visitor Pattern

- The control flows back and forth between the `visit` methods in the `Visitor` and the `accept` methods in the object structure.

```
class SumVisitor implements Visitor {  
    int sum = 0;  
    public void visit(Nil x) {}  
    public void visit(Cons x) {  
        sum = sum + x.head;  
        x.tail.accept(this);  
    }  
}  
List l;  
...  
SumVisitor sv = new SumVisitor();  
l.accept(sv);  
System.out.println(sv.sum);
```



Visitor Pattern: Everything together

```
interface List {  
    void accept(Visitor v);  
}  
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```

```
class SumVisitor implements Visitor {  
    int sum = 0;  
    public void visit(Nil x) {}  
    public void visit(Cons x) {  
        sum = sum + x.head;  
        x.tail.accept(this);  
    }  
}  
List l;  
...  
SumVisitor sv = new SumVisitor();  
l.accept(sv);  
System.out.println(sv.sum);
```

```
class Nil implements List {  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}  
class Cons implements List {  
    int head;  
    List tail;  
    public void accept(Visitor v) {  
        v.visit(this);  
    }  
}
```

Several compilers use
visitor pattern to perform
syntax-directed translation.



One wrinkle in SumVisitor

```
class SumVisitor implements Visitor {  
    int sum = 0;  
    public void visit(Nil x) {}  
    public void visit(Cons x) {  
        sum = sum + x.head;  
        x.tail.accept(this);  
    }  
}
```

- `visit` returns `void`. The actual result lives in a mutable field `sum`.
 - We are bringing effects through side-effects — not so nice.
- What if a different visitor needed to return a `String`? Or a `boolean`?
 - Different field, different Visitor interface, every single time — no reuse at all.



The same problem, in a different costume

```
List list = new ArrayList();

list.add("Hello");
list.add(42);                                // Allowed!

String s = (String) list.get(0);             // OK
String t = (String) list.get(1);             // Run-time exception
```

- A raw list does not know what it holds — every `get()` needs an unchecked cast, and the programmer may have mixed types that may lead to run-time exceptions.
- Generics let a class or a method be parameterized by a type:

```
List<String> words = new ArrayList<>();

words.add("Hello");
// words.add(42);                                // Compile-time error

String s2 = words.get(0);                       // No cast needed
```



Defining a generic class

```
class Stack<T> {  
    List<T> elements = new ArrayList<>();  
    void push(T item) {  
        elements.add(item); }  
    T pop() {  
        return elements.remove(elements.size() - 1); }  
}  
  
Stack<Integer> intStack = new Stack<>();  
intStack.push(10);  
intStack.push(20);  
System.out.println(intStack.pop());    // 20  
  
Stack<String> stringStack = new Stack<>();  
stringStack.push("CS339");
```

OO polymorphism: subtype-based, resolved at run time.
Generic polymorphism: parametric, enforced at compile time.



Why not just extend `java.lang.Object`?

```
class ObjectStack {
    List<Object> elements = new ArrayList<>();
    void push(Object o) {
        elements.add(o); }
    Object pop() {
        return elements.remove(elements.size() - 1); }
}

ObjectStack os = new ObjectStack();
os.push("CS339");
os.push(42); // Allowed!
String s = (String) os.pop(); // Run-time cast, may fail
```

- Same problem as before: type safety is quietly thrown away.
- **Inheritance alone cannot express “a stack of exactly one type, whichever type I choose.” Only a genuine type parameter can.**



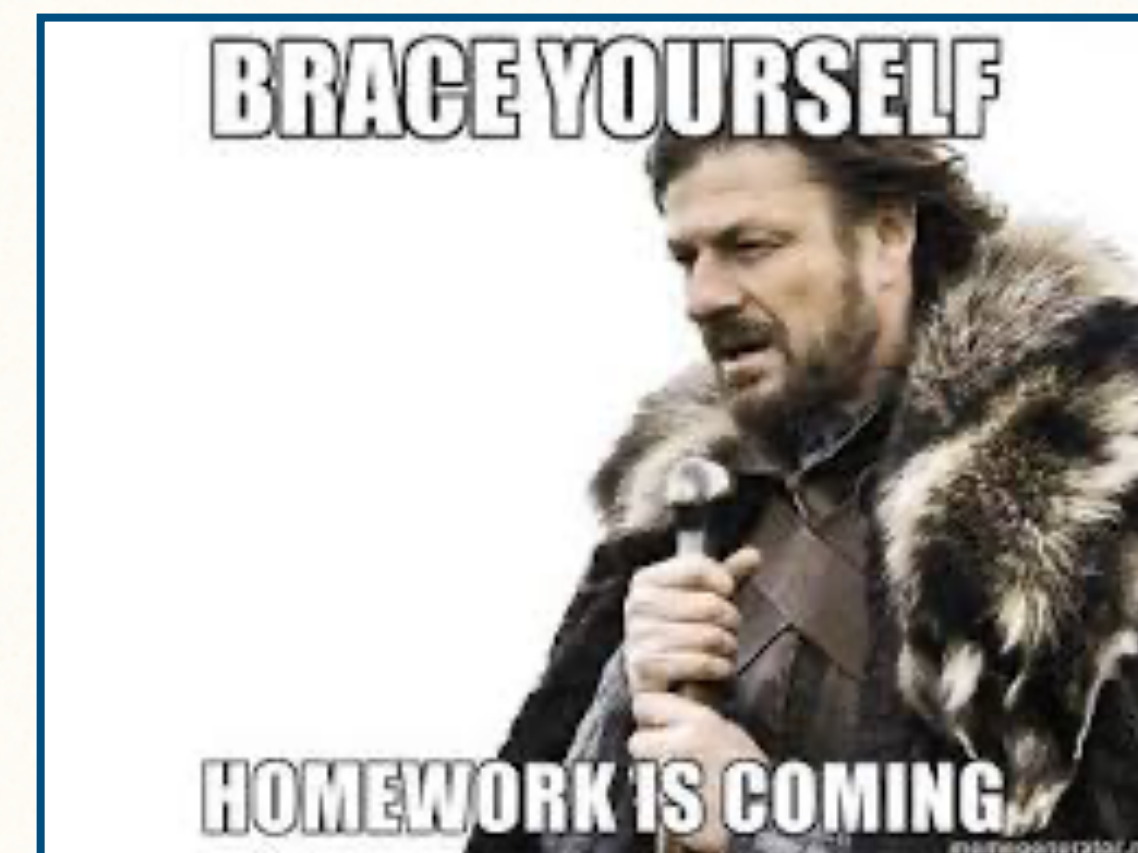
Java Generics versus C++ Templates

- Java compiles generics via type erasure — a single class is generated, and the type parameter is erased at run-time:

```
List<String> ls = new ArrayList<>();  
List<Integer> li = new ArrayList<>();  
System.out.println(ls.getClass() == li.getClass()); // true
```

- C++ templates go the other way: the compiler generates a fresh class for every instantiation — `vector<int>` and `vector<string>` are genuinely different types at run time.

➤ Comparison?



Back to Visitors: generifying accept

Before:

```
interface List {  
    void accept(Visitor v);  
}  
  
interface Visitor {  
    void visit(Nil x);  
    void visit(Cons x);  
}
```

After:

```
interface List<R> {  
    R accept(Visitor<R> v);  
}  
  
interface Visitor<R> {  
    R visit(Nil x);  
    R visit(Cons x);  
}
```

- <R> before the return type declares a fresh type parameter, local to this one call — different callers can pick different R, all still checked at compile time.



SumVisitor, without the mutable field

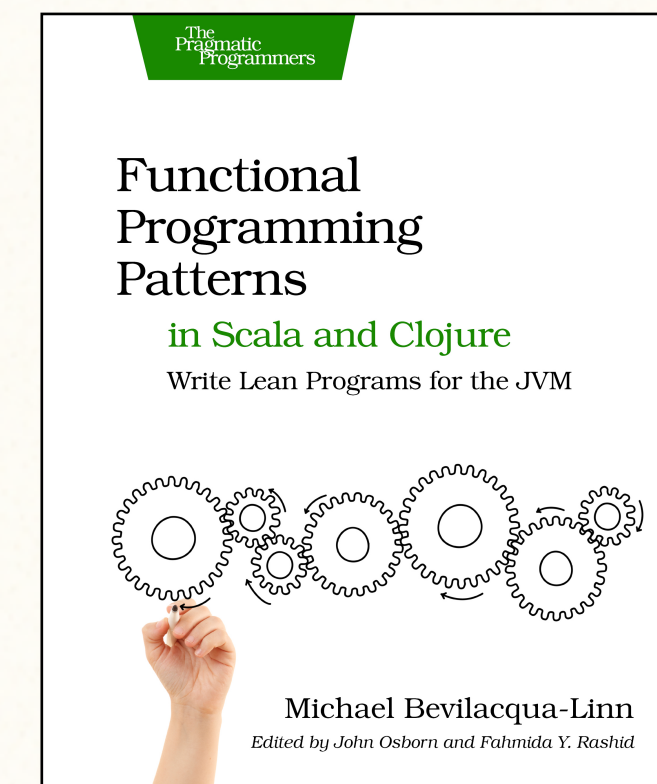
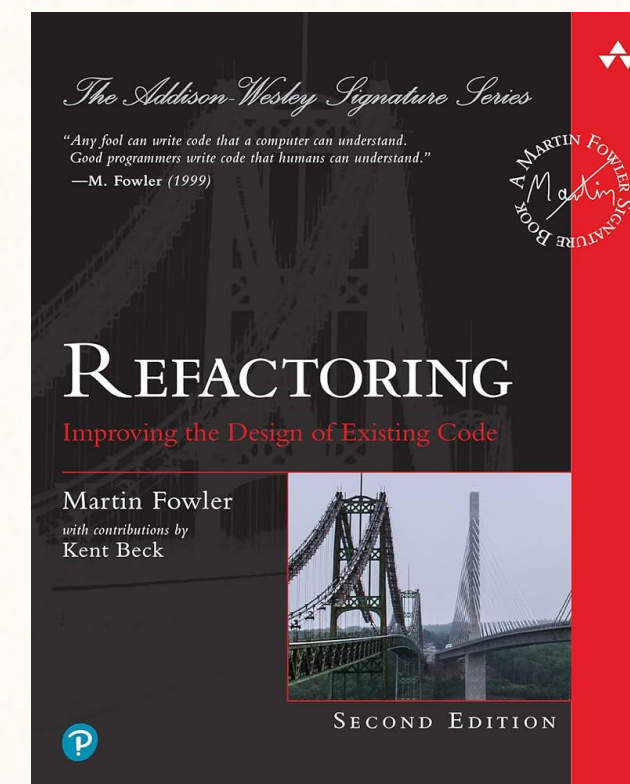
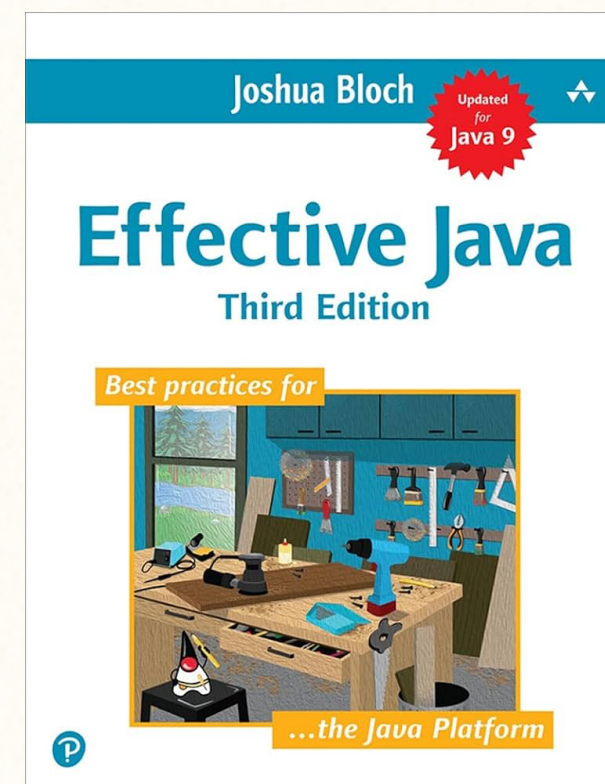
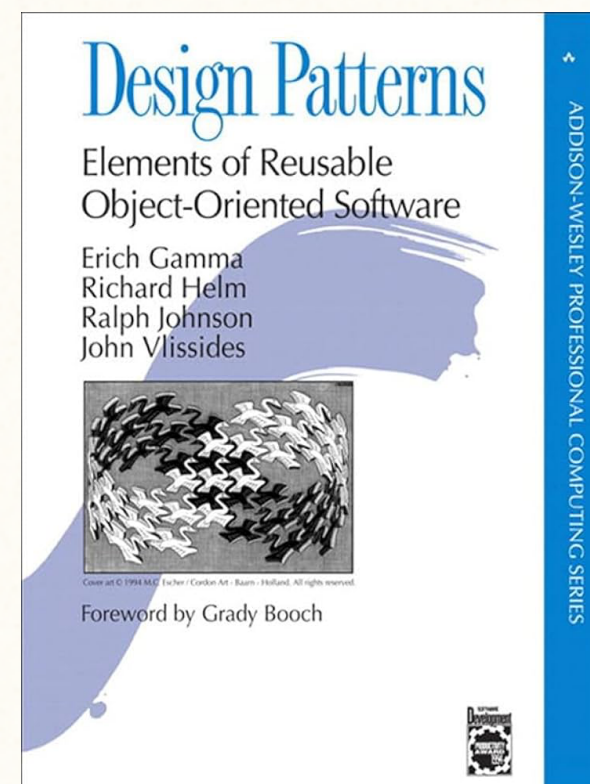
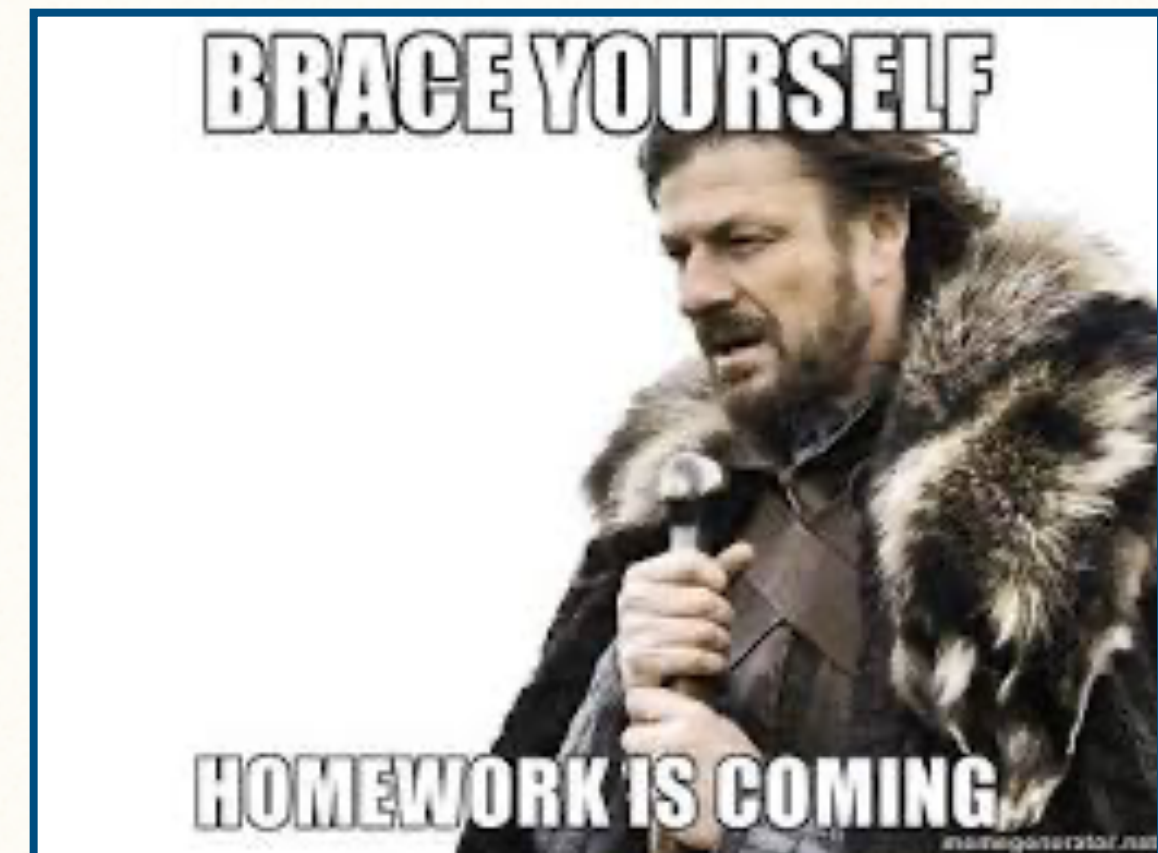
```
class SumVisitor implements Visitor<Integer> {  
    public Integer visit(Nil x) { return 0; }  
    public Integer visit(Cons x) {  
        return x.head + x.tail.accept(this);  
    }  
}  
  
List l = ...;  
int sum = l.accept(new SumVisitor());
```

- No mutable field. No cast. The compiler guarantees a `SumVisitor` can only ever be paired with something that checks out as `Integer`.



For some other day

- There are several more interesting OO patterns that help solve problems elegantly
- Particularly notable ones (find out as another **homework**):
 - *Strategy Pattern*
 - *Decorator Pattern*
 - *Factory-method Pattern*
- Learn more (not in this course):



Hello, World!

```
class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

- When you type `java Main` at the prompt:
 - The JVM internally invokes `Main.main()`
 - which is a public *static* method (i.e., accessible to the JVM directly through the class)
 - and supplies any passed command-line arguments through the `String` array `args`
- `out` is a *static* field of type `java.io.PrintStream` in the `java.lang.System` class
- and `println` is a method in the `PrintStream` class!



Unevaluated...



Next class: a new way of thinking about writing stateful programs; and we finally evaluate the “Unevaluated”.